

本科毕业论文（设计）

基于 RAG 与工作流的中原工学院招生咨询系统

Retrieval-Augmented Workflow-Based Enrollment Consultation System for Zhongyuan University of Technology

学院名称： _____

专 业： _____

班 级： _____

学 号： _____

学生姓名： _____

指导教师： _____

2026 年 5 月

诚信承诺书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所取得的成果。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或在网上发表的论文。

论文作者签名：_____

日期：_____

摘 要

中原工学院招生咨询业务同时面向考生、家长与招生工作人员，问题类型覆盖招生章程、专业目录、录取分数线、学费资助、学院介绍和报到流程等多个方面。传统静态问答库虽然实现成本低，但难以处理自然语言表达差异、跨文档信息整合和多轮上下文承接等问题；纯大模型对话又容易在高风险招生场景中产生事实幻觉，难以满足“回答可追溯、结果可验证、过程可管理”的业务要求。针对上述问题，本文围绕真实项目实施，设计并完成了基于检索增强生成（Retrieval-Augmented Generation, RAG）与 workflow 编排的中原工学院招生咨询系统。

系统采用前后端分离与后端网关编排相结合的总体架构。前端基于 Vue 3、TypeScript 与 Vite 构建多会话咨询界面，支持普通对话、RAG 问答和专家模式切换，以及 SSE 流式输出。后端以 FastAPI 为统一入口，结合 LangGraph 工作流，将查询改写、混合检索、结构化招生查询、引用校验、答案生成、技能执行和故障降级组织为可追踪的服务链路。数据层同时维护非结构化招生文档与结构化招生知识表：前者通过 PDF、DOCX、XLS/XLSX 解析后构建 FAISS 向量索引和 BM25 稀疏索引，后者使用 MySQL 存储专业目录、分数线和政策附表，从而兼顾开放性问答与精确字段查询。

在关键算法上，本文设计了“稠密检索 + 稀疏检索 + 精确匹配 + RRF 融合 + 重排 + 质量闸门”的 RAG 检索链路，并引入基于工作流的可插拔能力编排机制，用于解决复杂问题中的步骤规划、证据组织和依赖故障隔离。针对招生业务的严肃性要求，系统进一步加入引用校验、冲突裁决、追踪标识、健康检查和指标上报等工程措施，以降低大模型回答的不确定性。

基于项目现有评测结果，本文从功能正确性、检索效果、问答质量与容错能力四个维度进行了系统测试。招生 RAG 专项评测覆盖 70 个样本，最终答案通过率达到 100%，工具路由准确率达到 100%，平均总分为 0.9933；问答质量抽样评测中，关键词覆盖率与引用命中率均为 100%，幻觉率为 0；在 5 组故障注入型网关回放测试中，系统均返回有效响应，P95 延迟为 13.98 ms，说明网关编排与降级链路能够维持核心服务可用。测试结果表明，该系统已经具备面向高校招生咨询场景的落地基础，但在 RAG 检索排序质量、跨实例记忆持久化以及部署架构收敛方面仍有进一步优化空间。

本文工作说明，将 RAG 与 workflow 编排结合，并针对高校招生领域构建结构化与非结构化协同知识底座，能够在保证回答准确性与可追溯性的同时提升系统的工程可维护性与扩展能力，对同类垂直问答系统设计具有一定参考价值。

关键词：RAG；workflow 编排；招生咨询；混合检索；智能问答系统

ABSTRACT

The enrollment consultation scenario of Zhongyuan University of Technology involves candidates, parents, and admission staff, with queries spanning admission policies, major catalogs, score lines, tuition, financial aid, school introductions, and registration procedures. Traditional FAQ systems are inexpensive but weak in handling natural language variation, cross-document evidence aggregation, and multi-turn context. Pure generative chat systems, on the other hand, are prone to factual hallucination in high-stakes admission services. To address these problems, this thesis designs and implements an enrollment consultation system that integrates Retrieval-Augmented Generation (RAG) with workflow-based orchestration.

The system adopts a front-end and back-end separated architecture with a gateway-oriented orchestration layer. The front end is built with Vue 3, TypeScript, and Vite, and supports multi-session interaction, mode switching, and Server-Sent Events based streaming responses. The back end uses FastAPI as the unified entry point and employs LangGraph workflows to organize query rewriting, hybrid retrieval, structured admission lookup, citation checking, answer generation, skill execution, and graceful degradation. At the data layer, unstructured admission documents are parsed into retrievable text and indexed through FAISS and BM25, while structured admission facts such as major information, score lines, and policy tables are stored in MySQL for precise field-level access.

The core retrieval chain combines dense retrieval, sparse retrieval, exact match, reciprocal rank fusion, reranking, and quality gating. A workflow-based orchestration mechanism is introduced to manage step planning, tool routing, evidence assembly, and dependency isolation. To improve trustworthiness in an admission setting, the system further integrates citation validation, conflict arbitration, trace

identifiers, health checks, and metrics reporting.

Empirical results based on the existing project evaluation reports show strong effectiveness. In the dedicated admission RAG evaluation with 70 cases, the final answer pass rate reached 100%, the tool routing accuracy reached 100%, and the average total score was 0.9933. In sampled answer-quality evaluation, keyword coverage and citation hit rate both reached 100%, while the hallucination rate remained 0. In five fault-injection based gateway replay cases, the system consistently returned valid responses with a P95 latency of 13.98 ms, indicating that the orchestration and degradation paths can preserve core service availability.

The study demonstrates that combining RAG with workflow orchestration, together with a structured and unstructured hybrid knowledge base, can improve answer accuracy, traceability, and maintainability in university admission consultation. The system still has room for improvement in retrieval ranking quality, persistent memory across instances, and deployment convergence, but it already provides a feasible engineering solution for vertical consultation scenarios.

Keywords: RAG; Workflow Orchestration; Enrollment Consultation; Hybrid Retrieval; Intelligent Question Answering System

目 次

摘要.....	1
第 1 章第章 引言.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 检索增强生成研究现状.....	2
1.2.2 代理式 workflows 研究现状.....	2
1.2.3 高校垂直问答系统研究现状.....	3
1.3 研究内容与目标.....	3
1.4 论文结构.....	4
第 2 章第章 相关技术与需求分析.....	6
2.1 RAG 技术基础.....	6
2.2 工作流与代理编排技术.....	6
2.3 混合检索与结构化查询技术.....	7
2.3.1 BM25 与向量检索.....	7
2.3.2 RRF 融合与重排.....	8
2.3.3 结构化招生查询.....	8
2.4 系统需求分析.....	9
2.4.1 功能需求.....	9
2.4.2 非功能需求.....	9
2.4.3 招生场景的特殊约束.....	9
第 3 章第章 系统总体设计.....	10
3.1 设计原则.....	10
3.2 总体架构设计.....	10
3.3 前端总体设计.....	11
3.4 后端总体设计.....	12
3.5 数据层设计.....	12
3.5.1 非结构化知识源.....	12
3.5.2 结构化招生知识库.....	12
3.5.3 部署预留组件与当前主链路.....	13
3.6 部署与运行设计.....	13
第 4 章第章 关键模块设计与实现.....	14
4.1 前端交互模块设计与实现.....	14
4.2 网关编排模块设计与实现.....	14

4.3	RAG 检索模块设计与实现.....	15
4.3.1	文档解析与索引构建.....	15
4.3.2	混合召回与质量控制.....	15
4.3.3	引用约束机制.....	16
4.4	结构化招生查询模块设计与实现.....	16
4.5	记忆与技能模块设计与实现.....	16
4.5.1	记忆模块.....	16
4.5.2	技能模块.....	17
4.6	可观测与容错模块设计与实现.....	17
第 5 章第章	系统测试与结果分析.....	18
5.1	测试环境与测试方案.....	18
5.2	功能测试分析.....	18
5.3	检索评测与问答质量评测.....	19
5.3.1	招生 RAG 专项评测.....	19
5.3.2	问答质量抽样评测.....	19
5.4	容错与可用性测试.....	20
5.5	测试结果分析与不足.....	20
第 6 章第章	结论与展望.....	22
6.1	研究结论.....	22
6.2	不足与展望.....	22
	参考文献.....	24
	附录.....	27
	附录 A 系统主要知识源示例.....	27
	附录 B 系统主要接口示例.....	27
	致谢.....	28

第 1 章 引言

1.1 研究背景与意义

随着高校招生宣传渠道不断线上化，考生和家长对招生信息的获取方式已经从纸质简章和热线电话逐步转向网站、公众号、短视频平台以及在线问答系统。招生咨询问题也从简单的“学校代码是什么”“某专业学费是多少”，扩展为“某省某专业的报考风险如何判断”“选科要求与培养方向如何理解”“新生报到流程与资助政策如何衔接”等更复杂的自然语言问题。仅依赖人工热线或静态网页，一方面难以承受高并发咨询压力，另一方面也难以在不同时段保持口径一致。

在这一背景下，基于大语言模型的智能问答系统为高校招生咨询提供了新的技术路径。大模型具备较强的自然语言理解与生成能力，能够对用户问题进行归纳、补全和多轮解释，但其核心短板也同样明显，即知识更新受训练数据时效限制，且在事实证据不足时容易生成貌似合理但实际错误的答案。招生咨询属于高风险业务场景，任何涉及录取规则、学费标准、选考要求和报到安排的错误回答，都可能直接影响考生决策，因而系统设计不能停留在“能聊”，而必须进一步满足“答得准、说得清、能追责”的要求。

检索增强生成技术为解决上述矛盾提供了较为有效的工程思路。**Lewis** 等提出的 **RAG** 框架通过在生成前动态检索外部知识，为模型输出提供依据和上下文支持 [1]。在此基础上，后续研究逐步从单一路径检索扩展为混合检索、主动检索、代理式检索与深度推理式 **RAG**[2, 7, 8]。与此同时，**ReAct** 等工作证明，将“推理”和“行动”耦合起来能够增强语言模型与外部工具、知识库和环境的交互能力 [6]。这些研究说明，对于复杂、跨来源、可验证的高校招生咨询任务，仅有检索或仅有生成都不够，还需要一个能够组织检索、工具、记忆、引用和容错的工作流式执行框架。

中原工学院招生资料具有来源多样、结构不一、时效敏感和字段约束明显等特征，既包含招生章程、报考指南、学院介绍等非结构化文档，也包含专业目录、录取

分数线、问答库和政策附表等结构化或半结构化数据。如何把这些异构资料转化为统一可检索、可引用、可持续维护的招生知识底座，并通过工作流式问答系统稳定对外提供服务，是本项目具有现实价值和研究价值的关键所在。

因此，本文选择“基于 RAG 与工作流的中原工学院招生咨询系统”作为研究对象，围绕高校招生智能问答的准确性、可追溯性和工程可落地性展开设计与实现，希望为垂直领域问答系统建设提供一套可复用的实践范式。

1.2 国内外研究现状

1.2.1 检索增强生成研究现状

RAG 已经成为缓解大语言模型知识静态化与幻觉问题的主流路线之一。Lewis 等较早将神经检索与生成模型整合，证明了外部知识注入对知识密集型 NLP 任务的有效性 [1]。随后，大量研究围绕检索策略、召回粒度、证据重排、生成控制和系统评测持续演进。Gao 等对大模型时代的 RAG 体系进行了系统梳理，指出数据源组织、索引构建、检索器设计、生成控制和评价方法是影响系统性能的关键因素 [2]。近年来，研究重点又从“检到什么”进一步转向“何时检、如何检、检后如何推理”，形成了更强调动态决策与多步骤推理的 Agentic RAG 路线 [9, 8]。

在检索层面，单一向量检索虽然能够较好应对语义相似问题，但在时间、数字、专有名词和表格式字段上仍存在召回不稳定的问题。BM25 及其扩展模型在关键词匹配和短文本定位方面仍具有较强优势 [3]，因此实际系统常采用向量检索与稀疏检索协同的混合检索方案，并使用 RRF 等融合方法对多路召回结果进行统一排序 [4]。在索引层面，FAISS 由于提供高效的近似向量相似搜索能力，被广泛用于本地部署型向量检索系统 [5]。

1.2.2 代理式工作流研究现状

在语言模型代理方向，ReAct 提出将推理链与行动指令交替生成，使模型能够在需要时调用外部工具获取信息并修正后续推理 [6]。这一思路对问答、决策和工具使用

任务都产生了较大影响，也为当前工作流式智能体框架奠定了方法基础。随着更长上下文和更强推理能力模型的出现，Search-o1 等研究开始探索“检索何时介入深度推理过程”，通过在不确定知识点处触发主动搜索，并对检索到的长文档进一步做文内推理，以提升复杂知识问答的稳定性 [7]。Yangning Li 等对 RAG-Reasoning 方向进行了综述，指出未来 RAG 系统的重要演进方向包括任务驱动的检索规划、证据级推理、执行轨迹可审计和多轮反馈闭环 [8]。

从工程视角看，代理式问答系统已经不再是“模型 + 检索器”的松散拼接，而是更加接近一个工作流系统：它需要根据问题类型决定是否走结构化工具、是否改写查询、是否重复检索、是否执行引用校验、是否输出降级提醒，甚至需要在依赖服务异常时切换策略。这说明智能问答系统的研究重点正在从模型单点能力，转向系统级协同能力与可运维能力。

1.2.3 高校垂直问答系统研究现状

高校场景下的问答系统长期存在，但传统实现多数依赖规则库、FAQ 检索或有限领域槽位填充，适合固定模板问题，却难以应对招生咨询这类既有政策条款、又有专业细节、还包含时效性更新的信息服务需求。对于高校招生业务而言，系统不仅要回答“是什么”，还需要清楚说明“依据来自哪里”，以保证用户对回答结果的信任。

另一方面，高校真实数据常常分散在章程、简章、专业表格、历史分数和学院宣传材料之中，不同来源的字段表达、发布时间和有效期并不一致。如果没有统一的数据处理与知识融合方案，系统要么会因为知识覆盖不足而答非所问，要么会因为多个来源冲突而给出不一致结论。因此，将结构化招生表与非结构化文档统一建模，并通过引用校验和工作流控制降低错误风险，是高校招生问答系统真正可用的关键。

1.3 研究内容与目标

本文的研究内容围绕一个真实的中原工学院招生咨询项目展开，重点不在于训练新模型，而在于将现有大模型能力、RAG 技术、结构化招生知识和工作流编排机制系

统化整合，构建出一套具备工程可用性的垂直问答系统。围绕这一目标，本文主要完成如下工作：

1. 设计面向招生场景的异构知识底座，对招生章程、报考指南、录取分数线、招生专业详情、学院介绍与 **FAQ** 等资料进行解析、清洗、分块、索引与结构化入库。
2. 设计基于 **RAG** 与 workflow 编排的后端执行链路，将查询改写、混合召回、重排、引用校验、结构化查询、记忆接入、技能调用和答案生成组织为可追踪、可降解的服务流程。
3. 构建面向咨询用户的前端交互界面，实现多会话管理、模式切换、**SSE** 流式输出和后台管理操作展示。
4. 基于现有评测报告，对系统从功能正确性、检索效果、问答质量和故障容忍性等角度进行分析，总结系统优点、现存不足与后续优化方向。

对应的研究目标包括：第一，提高招生咨询问答的事实准确性和字段精确性；第二，提升回答过程的证据可追溯性与结果可解释性；第三，在依赖服务异常或部分能力不可用时维持系统主链路稳定；第四，保证系统具备可扩展、可维护、可持续演进的工程结构。

1.4 论文结构

全文共分为六章，具体安排如下：

1. 第一章介绍研究背景、研究意义、国内外研究现状、本文研究内容与论文结构。
2. 第二章介绍系统设计涉及的核心理论与需求分析，包括 **RAG**、工作流代理、混合检索和招生业务需求。
3. 第三章从整体视角阐述系统总体设计，包括架构分层、数据层设计、交互设计与部署方案。
4. 第四章详细说明系统关键模块的设计与实现，包括前端交互、网关编排、**RAG** 检索、结构化查询、记忆与技能、可观测与容错机制。

5. 第五章结合项目评测报告对系统进行测试与结果分析。
6. 第六章总结全文工作，并提出系统后续优化方向。

第 2 章 相关技术与需求分析

2.1 RAG 技术基础

检索增强生成的核心思想，是在生成答案前从外部知识源动态检索与问题相关的证据，再将证据与用户输入共同送入生成模型，从而提高模型的事实可靠性与知识时效性 [1]。与直接依赖参数记忆的纯生成模型相比，**RAG** 将知识访问过程显式化，使系统能够在回答时利用最新资料，也便于对回答进行来源追踪。

针对本项目的招生咨询场景，**RAG** 的优势主要体现在三个方面。第一，招生政策和专业信息具有明显时效性，每年章程、专业目录和录取分数线都有更新，外部知识检索能够降低“旧知识答新问题”的风险。第二，招生问题中包含大量数字、年份、专业名称和学院名称等高精度字段，检索阶段可以优先定位原始依据，减少模型自由发挥。第三，招生咨询经常要求“说明依据”，**RAG** 允许系统同时返回证据片段与引用来源，更符合招生业务对于可追溯性的要求。

RAG 的性能取决于数据准备、索引构建、召回策略、重排质量和生成约束等多个环节。若数据切分过粗，模型可能得到大段噪声文本；若检索过度依赖语义相似，则数字类问题容易召回不准；若没有重排与质量闸门，即便召回了正确文档，也可能因证据排序偏后而影响最终回答。因此，实际工程中通常需要比“向量搜一遍再生成”更复杂的处理流程。

2.2 workflow 与代理编排技术

大语言模型在复杂任务中往往需要经历“理解问题、制定步骤、调用工具、读取结果、继续推理、整理回答”等多个阶段。若将所有逻辑硬编码为单一函数调用，系统很快会面临维护困难、扩展困难和异常处理困难的问题。工作流编排技术的价值，就在于把复杂问答过程拆分为多个可控步骤，并对每一步的输入、输出和失败策略进行显式管理。

ReAct 的研究表明，把推理与行动相结合，能够使模型在解决复杂问题时更加灵活和可解释 [6]。在此基础上，本文项目采用 LangGraph 风格的代理 workflows，把查询改写、记忆加载、结构化路由、检索调用、引用校验和答案生成组织为图式执行链路。与单一顺序调用相比，这种方式更适合处理高校招生问答中的分支逻辑，例如：

1. 当问题包含专业名称、学费、选科要求等明确字段时，优先路由到结构化查询工具；
2. 当问题需要政策解释或学院介绍时，优先进入 RAG 检索链路；
3. 当证据不足或引用校验失败时，可以执行降级提示或二次检索；
4. 当外部依赖故障时，可以通过熔断与隔离执行器保护主流程。

工作流编排的另一个价值在于可审计。对于招生咨询这类业务，仅给出最终答案并不足够，系统还应记录调用了哪些能力、检索到了哪些来源、是否发生降级以及本轮会话的追踪标识，以便开发、运维和答辩环节进行问题定位。

2.3 混合检索与结构化查询技术

2.3.1 BM25 与向量检索

招生咨询问题既包含“这个专业怎么样”一类语义开放问题，也包含“学费多少”“河南代码多少”“某专业选考科目是什么”这类字段明确问题。单独使用向量检索时，前者表现通常较好，后者则可能因数字和专有名词表示能力不足而出现偏差。BM25 模型在关键词权重和词项匹配方面具有稳定表现 [3]，尤其适合章程条款、学院名称、专业名称和政策术语的快速定位。

因此，本文项目没有采用单路检索，而是将向量检索、BM25 稀疏检索与精确匹配结合起来，形成混合检索链路。向量检索负责发现语义相近内容，BM25 提升关键词定位能力，精确匹配则服务于年份、金额、专业代码、学院名称等硬约束字段。

2.3.2 RRF 融合与重排

混合检索带来的问题是，多路召回结果在评分空间上不可直接比较。RRF 融合通过将不同排序结果映射为基于名次的可比较分值，能够较稳定地整合多路召回列表 [4]。其基本思想可以表示为：

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (2-1)$$

其中， R 表示不同检索器的结果集合， $\text{rank}_r(d)$ 表示文档 d 在某一检索器结果中的排序位置， k 为平滑常数。该方法的优点在于不要求各检索器得分分布一致，适合工程场景中的异构召回融合。

对融合后的候选结果，系统还需要进行重排，以便将最适合回答当前问题的证据置于前列。对于招生政策类与学院介绍类问题，重排尤其重要，因为这些文档篇幅较长、噪声较多，仅依赖初始召回排序很难保证第一条证据就是最佳依据。

2.3.3 结构化招生查询

高校招生问题中存在大量适合结构化处理的内容，如专业代码、学制、学费、选考要求、录取分数线、省份、年份和政策附表等。如果这类问题仍然完全依赖文本检索，不仅效率较低，也容易出现定位不准的问题。为此，本文项目在 RAG 之外额外设计了结构化招生查询链路。

结构化查询的思路是：根据用户问题中的关键词和实体信息，先判定该问题属于专业目录查询、分数线查询还是政策附表查询，再由对应的 MySQL 表进行字段过滤和条件匹配。这样，系统既保留了自然语言交互入口，又能在字段确定的情况下获得接近数据库检索的精确结果。

2.4 系统需求分析

2.4.1 功能需求

结合项目目标与真实招生业务，系统至少需要满足以下功能需求：

1. 支持考生、家长和工作人员通过自然语言提出招生相关问题，并获得中文回答。
2. 能够处理招生章程、专业详情、分数线、学费资助、学院介绍、报到流程等多种问题类型。
3. 支持结构化字段查询与非结构化文档问答协同工作。
4. 支持证据引用展示，便于用户判断回答依据。
5. 支持多会话管理、流式输出和模式切换，优化交互体验。
6. 支持后台健康检查、索引重建与评测接口，方便维护。

2.4.2 非功能需求

招生咨询系统作为垂直业务系统，除功能正确性外，还应满足以下非功能需求：

1. 准确性：对学费、专业、分数线、代码等关键事实必须尽量保证精确。
2. 可追溯性：回答应尽量附带来源，避免“只给结论不给依据”。
3. 可扩展性：系统应支持后续增加新学院资料、新工具、新服务和新知识源。
4. 鲁棒性：当某一依赖服务故障时，不应导致整个系统完全不可用。
5. 可维护性：架构模块应职责清晰，便于后续迭代与问题定位。

2.4.3 招生场景的特殊约束

与开放域聊天不同，高校招生问答具有三类特殊约束。第一，信息错误成本高，任何与录取规则、学费、报名流程有关的错误都可能直接影响考生利益。第二，数据来源异构且更新频繁，系统必须区分“当前稳定主链路”和“部署预留组件”，不能把规划写成事实。第三，用户提问表达多样，既可能是一句完整自然语言，也可能只是“人工智能学院电话”“法学学费”这样的短查询，因此系统需要兼顾自然语言理解与精准实体匹配能力。

第 3 章 系统总体设计

3.1 设计原则

为满足前述需求，本文系统设计遵循以下原则：

1. 证据优先：对关键事实性问题，优先从招生知识源中检索或查询，再组织生成结果。
2. 结构与非结构协同：字段明确的问题优先走结构化查询，解释性问题优先走文档检索，以兼顾准确性与覆盖面。
3. 工作流可编排：问答过程中的检索、生成、校验、记忆和工具调用都应可以被编排、审计和降级。
4. 服务可拆分：在 MVP 阶段允许本地调用，在部署阶段支持能力按服务独立拆分。
5. 工程可运维：系统应具备健康检查、指标输出、评测报告和发布门禁能力。

3.2 总体架构设计

根据项目源码与分析文档，系统总体采用“前后端分离 + 网关编排 + 可插拔能力服务 + 本地知识增强”的分层结构 [15, 14]。其核心组成可以概括为表现层、接入层、编排层、能力层和数据层五部分。

表 3-1: 系统总体架构分层

分层	主要职责
表现层	负责多会话管理、用户输入、消息展示、流式输出与后台操作入口。
接入层	基于 FastAPI 提供统一 API，对外暴露聊天、流式聊天、健康检查和管理接口。
编排层	根据问题类型组织查询改写、RAG、结构化查询、技能调用、记忆接入和生成输出。
能力层	提供 RAG 检索、记忆管理、技能管理、生成服务和可观测服务等能力模块。
数据层	存储招生原始文档、FAISS 索引、BM25 索引、MySQL 结构化表及可选 Neo4j 图谱。

这种架构的一个显著优点，是把“业务接入”和“能力实现”解耦。前端无需感知检索与生成的具体细节，只需根据模式组合发送请求；后端网关也无需将所有能力写死在控制器内部，而是通过编排层调度各能力服务，从而为后续扩展提供了空间。

3.3 前端总体设计

前端采用 Vue 3、TypeScript 和 Vite 技术栈实现 [15]。整体界面可划分为左侧会话栏、中间主对话区、底部输入操作区和右侧管理面板四部分。通过组件化设计，前端分别使用 ChatMain、ActionBar、RightPanel、MessageBubble 和 StreamBubble 等模块组织页面逻辑。

在交互模式上，前端支持普通对话模式、RAG 问答模式和专家模式。不同模式会影响请求体中功能开关的组合：普通模式强调基础生成；RAG 模式启用检索与引用约束；专家模式在此基础上进一步开放子问题拆分、步骤规划和技能调用等编排

能力。需要强调的是，专家模式当前属于多能力编排型问答入口，其稳定主链路仍以本地 **RAG**、结构化查询、记忆与技能机制为主；**MCP** 外部工具与 `web_search` 更适合作为受控预留或按配置启用的扩展能力，而不宜表述为默认稳定的联网自治代理。

3.4 后端总体设计

后端入口位于 `backend/app/main.py`，框架选型为 **FastAPI**。系统对外提供聊天、流式聊天、功能元数据、记忆压缩、工具目录、技能管理和健康检查等接口 [15]。在后端内部，真正承载业务逻辑的是以 `gateway.py` 为核心的网关编排模块，以及 `agent_graph.py`、`agent_runtime.py` 等专家 workflow 运行模块。

后端设计的关键不是接口数量，而是其执行链条的组织方式。系统先完成请求清洗和功能路由，再根据配置决定是否加载记忆、调用 **RAG**、进入结构化查询、启用技能执行以及执行引用校验，最后通过同步或流式方式返回结果。对于外部依赖，系统同时支持进程内本地调用和基于 **HTTP** 的服务调用两种模式，以便兼顾开发调试与容器化部署需求。

3.5 数据层设计

3.5.1 非结构化知识源

系统的非结构化知识源主要包括中原工学院 2025 年普通本科招生章程、2025 报考指南、各学院介绍材料及招生网站抓取结果等文档 [11, 15]。这些数据以 **PDF**、**DOCX** 和网页转换文本等形式存在，需要先经过解析、清洗和分块，再构建面向 **RAG** 的检索索引。

3.5.2 结构化招生知识库

系统同时维护以 **MySQL** 为核心的结构化招生知识库。根据源码实现，数据表至少包含 `documents`、`major_catalog`、`score_lines`、`policy_tables`、`faq_seed` 和 `parse_runs` 等实体 [15]。这些数据分别对应原始文档记录、专业目录、录取分数线、

政策附表、FAQ 种子集和解析运行历史。

结构化知识库的存在，使系统可以在“某专业学费多少”“某省某专业录取分数线是多少”“某专业选考科目是什么”等场景下直接给出高精度结果，而不必完全依赖非结构化文本召回。

3.5.3 部署预留组件与当前主链路

从 `docker-compose.yml` 可见，系统部署方案中预留了 PostgreSQL/pgvector 与 Redis 容器 [14]。但结合当前代码主链路分析可知，稳定运行路径仍以 FAISS、本地内存和 MySQL 为主，Redis 与 PostgreSQL 更偏向未来演进预留，而非当前已形成闭环的核心依赖 [15]。同样，Neo4j 图谱增强与 MCP 外部工具也属于按配置启用的可选扩展，而不是当前每轮请求都会默认进入的稳定主链路。这一点在论文表述中必须如实说明，不能把部署预留或可选增强能力误写为当前已稳定落地能力。

3.6 部署与运行设计

系统支持本地运行与 Docker Compose 两种部署方式。前端通过 `npm run dev` 启动开发环境，后端通过 `python main.py` 启动 FastAPI 服务；在容器化方案中，前端、API 网关、RAG 服务、记忆服务、技能服务、生成服务和可观测服务均可独立运行 [14]。这种设计既适合开发阶段的快速联调，也适合后续在更正式环境中按服务拆分扩展。

第 4 章 关键模块设计与实现

4.1 前端交互模块设计与实现

前端的核心任务，是把复杂后端能力以尽可能简单直观的方式呈现给用户。系统使用单页应用架构维护当前会话、消息列表、流式增量文本和右侧面板状态。用户在输入问题后，前端会根据模式选择构建不同的请求体，并通过统一的 **API** 服务模块发起普通 **HTTP** 请求或 **SSE** 流式请求。

这一设计有两个直接好处。第一，前端与后端之间的能力耦合较低，功能切换主要依赖请求参数而非页面结构重构；第二，流式输出允许用户在后端生成尚未结束时就逐步看到回答内容，降低等待感。对于招生咨询场景，这种交互方式比一次性返回长文本更符合用户预期。

4.2 网关编排模块设计与实现

网关编排模块是整个系统的控制中枢。根据源码分析，`gateway.py` 负责统一受理聊天请求、组织功能路由、记录追踪标识 `trace_id`、聚合降级特征 `degraded_features`，并支持普通响应与流式响应两种输出路径 [15]。其设计思路不是“一个接口里堆满业务判断”，而是将编排职责独立出来，使各能力模块可以按需插拔。

从执行流程上看，一次典型咨询请求通常经历如下阶段：

1. 接收请求并生成会话级追踪标识；
2. 根据模式与问题内容判定需要启用的功能集合；
3. 装载短期记忆、长期摘要或特殊记忆；
4. 对用户问题进行必要的查询改写；
5. 根据问题路由到 **RAG**、结构化查询或技能执行链路；
6. 对结果执行引用校验与降级判断；
7. 组织最终答案，并以同步或 **SSE** 方式返回给前端。

这种网关式组织方式体现了单一职责与可扩展原则：生成模块只负责生成，记忆模块只负责记忆，检索模块只负责检索，而编排层负责把这些能力按业务逻辑串起来。

4.3 RAG 检索模块设计与实现

4.3.1 文档解析与索引构建

RAG 模块主要由 `ingest.py`、`index.py`、`retrievers.py`、`rerank.py`、`citation_guard.py` 和 `service.py` 等文件组成 [15]。在离线侧，系统通过解析脚本将 PDF、DOCX、XLS 和 XLSX 格式的招生资料统一转化为可检索文本或结构化记录，再按块切分构建索引。

向量检索部分基于 FAISS 实现 [5]，其优势在于本地可部署、查询速度快、适合当前规模下的招生资料相似度检索。稀疏检索部分基于 BM25 实现 [3]，用于补充向量检索在字段和关键词场景中的短板。对于年份、金额、院系名称和专业代码等强约束字段，系统还设计了精确匹配规则，提高检索的业务针对性。

4.3.2 混合召回与质量控制

从工程实现看，系统的 RAG 处理链路并非线性单步，而是至少包含查询标准化、查询改写、混合召回、RRF 融合、重排、冲突裁决、质量闸门检查、必要时的二次检索与引用校验等步骤 [15]。其中，RRF 的作用是把稠密、稀疏与精确匹配三路结果统一到一个排序空间；重排模型则进一步对候选证据进行相关性精修；质量闸门与引用校验负责在答案生成前后控制证据可靠性。

为了适应招生业务中的时间敏感问题，系统还在检索阶段增加了年份与有效期偏置逻辑，使“最新章程”“2025 年录取情况”这类问题能够优先命中当前有效资料，而不是被旧年份信息干扰。这一处理对招生场景尤其重要，因为不同年度政策在个别细节上可能存在差异。

4.3.3 引用约束机制

在招生咨询场景中，“给出依据”与“答案正确”同样重要。为此，系统在 RAG 模块中引入了 `citation_guard`，用于检查本轮回答是否满足最小来源数、首条来源得分或其他证据约束 [14]。若证据不足或校验失败，系统会将相应特征标记为降级，并在前端结果中显式提示。这种做法虽然会增加一部分工程复杂度，但能够有效抑制“没有依据也硬答”的风险。

4.4 结构化招生查询模块设计与实现

招生咨询中大量问题本质上属于数据库检索问题。系统在 `admissions_kb/router.py` 中对问题进行类型识别，在 `repository.py` 中完成对应表的查询执行 [15]。例如，分数线、位次和批次问题会路由到 `score_lines`；学费、学制、专业代码和选考要求问题会路由到 `major_catalog`；政策附表相关问题则会路由到 `policy_tables`。

这一模块的关键价值在于，把最容易引发误差的字段类问题从生成式链路中“拎出来”，交给更确定的结构化查询处理。查询结果既可以直接组织为答案，也可以作为结构化证据再次进入 RAG 上下文，供生成模块整合表达。这样既保留了自然语言问答的灵活性，又不丢失数据库查询的精确性。

4.5 记忆与技能模块设计与实现

4.5.1 记忆模块

记忆模块由 `memory.py` 实现，当前采用进程内存方式维护短期记忆、长期记忆与特殊记忆 [15]。从代码逻辑可见，短期记忆默认设置过期时间，长期记忆以 `rolling_summary` 方式持续累积摘要，特殊记忆可用于记录用户偏好等长期约束。该机制足以满足单实例 MVP 场景下的多轮对话承接需求。

但也必须看到，这种实现仍属于 MVP 级方案。若系统未来扩展到多实例部署，进程内字典无法在实例间共享状态，届时需要引入真正的外部缓存或数据库以实现

持久化与跨实例同步。

4.5.2 技能模块

技能模块由 `skill_manager.py` 等组件负责，支持技能保存、版本号生成、内容哈希去重和最优版本激活等能力 [15]。对于招生场景而言，技能机制意味着系统未来可以将费用资助解读、报到流程说明、志愿填报建议等专题能力封装为更稳定的工作流程模板，而不必每次都从零开始推理。

从当前项目阶段看，技能机制已经具备框架能力，但其执行方式仍以规则化模板和受控工作流为主，后续若要进一步提升复杂问题处理能力，需要结合更细粒度的任务规划、反馈学习与效果评估机制。

4.6 可观测与容错模块设计与实现

由于系统依赖多个能力模块和外部模型接口，一旦某一服务出现波动，就可能影响整体体验。为此，项目设计了隔离执行器、熔断机制、健康检查、管理指标接口以及输入输出安全审查链路 [15]。在代码层面，`isolation.py` 用于封装隔离执行与容错控制，`/healthz` 与 `/healthz/dependencies` 接口用于对外报告服务与依赖状态，`/api/admin/metrics` 提供基础管理指标，而网关还会对越权提示、提示词泄露和疑似敏感内容进行规则拦截。

系统的容错策略并不是“失败就报错结束”，而是尽量保证核心流程继续返回可用结果。其保底链路至少包含四层：当重排服务不可用时回退到启发式排序；当向量检索或远程 **Embedding** 不稳定时回退到本地相似度方案；当结构化数据库不可用或无结果时回退到本地缓存检索；当专家模式最终生成超时或部分步骤失败时，回退到基于已完成步骤的规则化保守汇总。同时，当某一辅助能力不可用或引用校验无法满足要求时，系统会记录降级特征并显式提示证据约束状态。此类设计对真实招生场景非常必要，因为用户通常更希望获得一个带风险提示的可用答案，而不是直接得到一次服务失败。

第 5 章 系统测试与结果分析

5.1 测试环境与测试方案

为了验证系统在真实招生咨询场景中的可用性，项目从功能测试、检索评测、问答质量评测和故障注入测试四个层面开展验证。测试数据主要来自 2025 年招生章程、招生专业详情、录取分数线、学院介绍和问答库等知识源；测试脚本则依托项目内的 `evaluate_gateway.py`、`run_rag_eval.py` 和其他评测脚本完成 [14, 16, 17]。

从代码仓库结构统计看，项目当前包含 23 个后端 Python 测试文件，并配套 6 个前端 Vitest 测试文件，覆盖 API、RAG 流程、检索消融、发布门禁、隔离与并发、知识库构建、请求构造与界面组件等模块。虽然测试覆盖并不等于系统完全无缺陷，但这说明项目并非停留在功能演示层面，而是已经初步形成“自动化测试 + 评测脚本 + 发布门禁”的研发质量闭环。

5.2 功能测试分析

功能测试重点检查以下内容：聊天接口是否可用、流式输出是否稳定、模式切换是否生效、结构化查询能否正确命中、引用展示是否返回、后台管理能力是否可调用。根据项目运行结果，系统已支持普通聊天和流式聊天两种入口，并能返回包含 `status`、`degraded_features`、`sources` 和 `trace_id` 的完整结果结构 [14]。

此外，系统还能够响应健康检查、记忆压缩和索引重建类接口，说明其不仅面向最终用户，也考虑了运维与管理需求。对于毕业设计而言，这一点很关键，因为它体现了系统从“功能实现”向“可维护系统”迈进的工程意识。

5.3 检索评测与问答质量评测

5.3.1 招生 RAG 专项评测

根据 backend/reports/rag_eval_report.md 中的专项评测结果，系统共评测 70 个招生问答样本，全部样本最终通过，通过率为 100%，平均总分为 0.9933[16]。其中，工具路由准确率为 100%，Tool-first 首轮通过率为 100%，RAG-first 首轮通过率为 100%。这些结果表明，系统在“该走工具还是该走检索”的路由决策上表现稳定。

表 5-1: 招生 RAG 专项评测汇总

样本总数	通过数	通过率	平均总分	Tool 路由准确率
70	70	100%	0.9933	100%

需要注意的是，尽管最终答案全部通过，检索层面的 Recall@5、MRR@5 与 nDCG@5 平均值仅为 0.4286，说明系统在“最终能不能答对”与“前几条证据是否已被最佳排序”之间仍存在差距 [16]。换句话说，系统已经具备较强的结果正确性，但在检索排序效率与证据纯净度方面仍有优化空间。

5.3.2 问答质量抽样评测

根据 reports/answer_eval_report.json 的抽样评测结果，系统在 3 个问答用例中全部通过，关键词覆盖率与引用命中率均达到 100%，幻觉率为 0；在 2 个高难样本中同样全部通过 [18]。这说明系统至少在当前抽样范围内，能够保持较好的答案一致性和引用约束效果。

表 5-2: 问答质量抽样评测汇总

样本数	通过率	关键词覆盖率	引用命中率	幻觉率
3	100%	100%	100%	0

5.4 容错与可用性测试

项目还对网关层做了故障注入型回放测试。根据 `backend/reports/eval_report.json`，共执行 5 组测试，包括默认 RAG、受控预留的 Web 搜索降级、技能降级、已保存技能缺失和生成失败等场景，5 组用例全部返回有效响应，整体通过率为 100%，P95 延迟为 13.98 ms[17]。这里的延迟并不代表生产环境整体性能上限，而是在故障回放测试条件下网关处理链路的响应表现。

表 5-3: 网关故障注入回放测试结果

测试场景	是否返回有效结果	状态	延迟/ms
default_rag	是	degraded	24.97
web_search_degrade	是	degraded	13.98
skill_degrade	是	degraded	12.78
saved_skill_missing	是	degraded	10.39
generation_fail	是	failed	10.21

该结果说明，系统在面对部分依赖故障时，能够通过降级或失败封装维持接口层稳定，不会因为单点能力不可用而直接导致服务崩溃。对于招生咨询系统而言，这种“尽量返回可用结果”的设计价值高于单纯追求所有能力都同时工作。

5.5 测试结果分析与不足

综合上述测试，可以得到以下结论：

1. 系统在最终答案正确率、工具路由准确率和引用命中率方面表现较好，证明“结构化查询 + RAG + workflow编排”的路线对招生咨询任务是有效的。
2. 结构化问题场景表现最稳定，说明把专业目录、分数线和政策附表沉淀为 MySQL 表的设计是正确的。
3. RAG 类问题虽然最终答案正确，但检索排序指标偏低，说明系统当前更像“能

找到对答案”，而不是“能最快把最优证据排在第一位”。

4. 记忆模块、Redis/PostgreSQL 预留组件与真正跨实例部署需求之间仍存在差距，表明当前系统更适合单实例或轻量部署环境。

因此，后续优化方向应集中在三点：第一，继续优化重排模型、实体约束和条款级切片，提高 RAG 检索排序质量；第二，将记忆持久化和部署层组件真正打通，提升系统的多实例扩展能力；第三，增强技能模块与代理规划能力，使复杂招生问题能够以更稳定的专题流程进行处理。

第 6 章 结论与展望

6.1 研究结论

本文面向中原工学院招生咨询业务，设计并实现了一套基于 **RAG** 与 workflow 编排的智能问答系统。系统通过统一接入层、可插拔编排层、混合检索链路、结构化招生知识库和前端流式交互界面的协同，较好地解决了高校招生场景中“问题表达自然、知识来源异构、事实字段精确、回答必须可追溯”的实际难题。

在系统设计上，本文将非结构化 **RAG** 与结构化查询结合起来，使解释型问题和字段型问题都能找到合适的处理路径；将 workflow 编排引入招生问答流程，使记忆接入、检索调用、技能执行、引用校验和降级策略具备明确边界和可审计性；将评测脚本、健康检查和指标输出纳入整体工程结构，使系统不仅能演示功能，也具备一定的运维与质量保障能力。

在实验与评测上，现有项目结果显示系统在 70 个招生问答样本上实现了 100% 的最终通过率，工具路由准确率达到 100%，问答抽样测试中的引用命中率达到 100%，故障注入场景下也能维持有效响应。这些结果表明，本文方案对高校招生咨询这一垂直业务场景具备现实可行性。

6.2 不足与展望

尽管系统已形成较完整的原型，但仍存在若干不足。第一，**RAG** 检索排序指标仍有提升空间，尤其是在政策条款和学院介绍类问题中，噪声证据较多，实体约束与重排策略还需优化。第二，记忆模块目前主要基于进程内存实现，尚未完成跨实例共享与持久化改造。第三，部署方案中预留的 **Redis** 与 **PostgreSQL/pgvector** 组件尚未与当前主链路完全打通，**Neo4j** 与 **MCP** 外部工具能力也仍主要停留在按配置启用的可选增强层，系统架构与部署架构仍需进一步收敛。第四，技能机制虽然具备版本管理与执行基础，但在复杂任务的规划编排稳定性方面仍有较大提升空间。

未来工作可从以下几个方向展开：

1. 基于招生政策条款、学院实体和年份信息构建更细粒度的重排与过滤机制，提高证据排序质量。
2. 引入外部缓存或数据库实现记忆持久化，并完善多实例下的状态管理方案。
3. 在验证业务收益与运维成本的前提下，逐步统一部署架构与实现架构，按需推动 Redis、PostgreSQL/pgvector 等预留组件进入真实业务闭环。
4. 丰富专题技能链路，将资助政策解读、志愿填报指导和报到流程服务进一步产品化。
5. 在严格控制风险的前提下，引入更强的主动检索与深度推理机制，提升复杂招生问题的处理能力。

总体而言，本文工作证明，将 **RAG** 与 workflow 编排结合，并围绕真实招生资料构建结构化与非结构化协同知识底座，能够形成一套兼顾准确性、可追溯性和工程可用性的高校招生咨询系统，为同类垂直场景的智能问答建设提供了有价值的实践参考。

参考文献

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[EB/OL]. arXiv:2005.11401, 2020.
- [2] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[EB/OL]. arXiv:2312.10997, 2023.
- [3] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.
- [4] Cormack G V, Clarke C L A, Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods[C]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2009: 758-759.
- [5] Johnson J, Douze M, Jégou H. Billion-Scale Similarity Search with GPUs[EB/OL]. arXiv:1702.08734, 2017.
- [6] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. arXiv:2210.03629, 2023.
- [7] Li X, Dong G, Jin J, et al. Search-o1: Agentic Search-Enhanced Large Reasoning Models[C/OL]//Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. 2025. Available: arXiv:2501.05366.
- [8] Li Y, Zhang W, Yang Y, et al. Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs[EB/OL]. arXiv:2507.09477, 2025.
- [9] Singh A, Ehtesham A, Kumar S, et al. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG[EB/OL]. arXiv:2501.09136, 2025.
- [10] 中原工学院教务处. 关于印发中原工学院本科毕业论文（设计）工作规范（修订）的通知 [Z]. 2026.
- [11] 中原工学院. 中原工学院 2025 年普通本科招生章程 [Z]. 2025.

- [12] 中原工学院. 2025 年招生专业详情 [Z]. 2025.
- [13] 中原工学院. 2025 年录取分数线 [Z]. 2025.
- [14] Enrollment-Consultation-System 项目组. Enrollment-Consultation-System README[Z]. 2026.
- [15] Enrollment-Consultation-System 项目组. 系统架构与技术栈分析、毕业设计正文素材、技术栈证据清单 [Z]. 2026.
- [16] Enrollment-Consultation-System 项目组. 中原工学院招生 RAG 评测报告 [Z]. back-end/reports/rag_eval_report.md, 2026.
- [17] Enrollment-Consultation-System 项目组. 网关评测结果报告 [Z]. back-end/reports/eval_report.json, 2026.
- [18] Enrollment-Consultation-System 项目组. 问答质量评测报告 [Z]. reports/answer_eval_report.json, 2026.

附录

附录 A 系统主要知识源示例

系统当前用于招生咨询的核心知识源主要包括：中原工学院 2025 年普通本科招生章程、2025 报考指南、2025 年招生专业详情、2025 年录取分数线、各学院招生宣传材料以及问答库等。这些资料经过文档解析、字段抽取、结构化入库和索引构建后，共同构成系统的数据底座。

附录 B 系统主要接口示例

系统后端提供的主要接口包括：`/api/chat`、`/api/chat/stream`、`/api/features`、`/api/mcp/tools`、`/api/skills/saved`、`/api/memory/compress`、`/api/admin/metrics`、`/healthz` 和 `/healthz/dependencies`。这些接口覆盖了问答、流式通信、功能配置、后台管理和运维检测等需求。

致谢

在本次毕业论文（设计）撰写与系统实现过程中，相关项目资料、模板规范与技术分析文档为本文提供了直接支撑。